MedVoice AI
Vapi Integration Guide

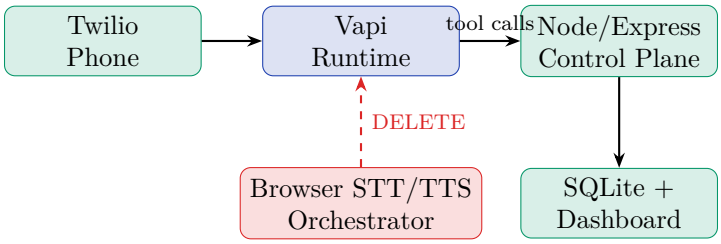
Technical Architecture

June 2026

Abstract

This document defines **exactly** what to build, keep, and remove when replacing MedVoice's hand-built real-time orchestration with Vapi as the live-call runtime. Vapi replaces only the middle box—not your product layer.

1 What Changes vs. What Stays



1.1 Drop (no longer own)

- `js/speech-io.js` — browser STT/TTS loop for phone calls
- `server/routes/voice.js` — TwiML ConversationRelay + WebSocket scaffold
- Real-time barge-in, endpointing, turn-taking, media streaming for *phone*
- Browser-only webhook delivery dependency for phone calls

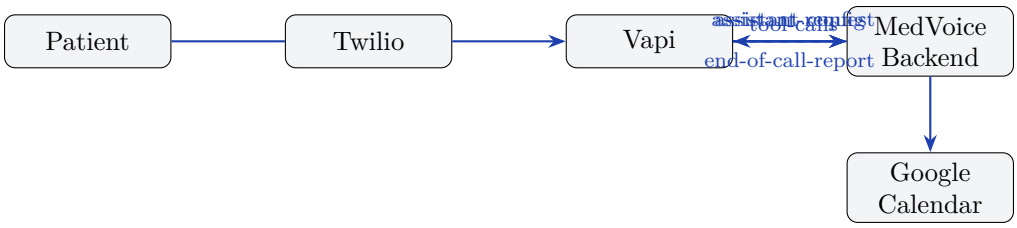
1.2 Keep and extend

- `configs/*.json` — per-tenant prompts, services, emergency keywords
- `server/routes/calendar.js` — becomes Vapi tool endpoints
- `server/routes/leads.js`, `sessions.js`, `emergency.js`
- `js/services/webhook-dispatcher.js` pattern — inbound from Vapi instead
- React dashboard, SQLite schema, multi-tenant `client_id` routing

2 The Two Integration Wires

Only two HTTPS connections link your product to Vapi:

1. **Configuration (you → Vapi)** — at call start, return tenant-specific assistant config.
2. **Callbacks (Vapi → you)** — tool calls during the call; end-of-call transcript/recording webhook.



3 Step-by-Step Implementation

3.1 Phase A: Accounts and keys (Day 1)

1. Create Vapi org at https://dashboard.vapi.ai; obtain `VAPI_API_KEY`.
2. Add to `.env`: `VAPI_API_KEY`, `VAPI_WEBHOOK_SECRET`.
3. Provision Twilio number; in Vapi, import number (BYO Twilio) or buy via Vapi.
4. Add provider keys in Vapi dashboard (BYOK): Deepgram STT, OpenAI/Groq LLM, ElevenLabs TTS.
5. Sign BAAs: Vapi + Twilio + Deepgram + OpenAI + ElevenLabs (HIPAA path).

3.2 Phase B: New backend routes (Days 2–4)

Create `server/routes/vapi.js` with three handlers:

3.2.1 POST /api/vapi/assistant-request

Triggered on inbound call. Map `call.phoneNumberId` or dialed `to` number → tenant.

Input (from Vapi):

```
{ "message": { "type": "assistant-request",
  "call": { "id": "...", "phoneNumber": { "number": "+1..." }}}}
```

Your logic:

1. Lookup tenant by dialed number in DB or `configs/`.
2. Load `configs/{client}.json`; build system prompt from services, greetings, emergency rules.
3. Respond within **7.5s** (telephony hard limit).

Output:

```
{ "assistant": {
    "name": "Aria",
    "firstMessage": "Hi, I'm Aria at MedVoice Clinic...",
    "model": { "provider": "openai", "model": "gpt-4o-mini",
      "messages": [{ "role": "system", "content": "<tenant prompt>" }],
      "toolIds": ["tool_check_avail", "tool_book", "tool_capture_lead"] },
    "voice": { "provider": "11labs", "voiceId": "<tenant voice>" },
    "serverUrl": "https://your-domain.com/api/vapi/webhook"
}}
```

3.2.2 POST /api/vapi/webhook (tool-calls)

Input:

```
{ "message": { "type": "tool-calls",
  "toolCallList": [{ "id": "toolu_xxx", "name": "check_availability",
    "arguments": { "date": "2026-06-17", "time": "10:00" }}],
  "call": { "id": "call-uuid" }}}
```

Output (required format):

```
{ "results": [{ "toolCallId": "toolu_xxx",
    "result": "Tuesday 10:00 AM is available." }] }
```

Wire tools to existing routes:

- `check_availability` → `POST /api/calendar/check`
- `book_appointment` → `POST /api/calendar/book`
- `capture_lead` → `POST /api/leads`
- `log_emergency` → `POST /api/emergency`

Pass `client_id` via Vapi **static parameters** (server-trusted, invisible to LLM).

3.2.3 POST /api/vapi/webhook (end-of-call-report)

On `message.type === "end-of-call-report"`:

- Persist transcript to `sessions` + `turns` tables
- Store recording URL, duration, structured lead data
- Fire outbound webhook via server-side dispatcher (not browser)

3.3 Phase C: Vapi tool definitions (Day 3)

Register in Vapi dashboard or via `POST /tool`:

Tool	Parameters	Server URL
`check_availability`	date, time, duration	/api/vapi/tools/calendar-check
`book_appointment`	date, time, patientName, reason	/api/vapi/tools/calendar-book
`capture_lead`	name, phone, dob, insurance	/api/vapi/tools/lead
`transfer_urgent`	reason	/api/vapi/tools/transfer

3.4 Phase D: Phone routing (Day 4)

1. `PATCH /phone-number/{id}`: set `assistantId: null`, `server.url` = your assistant-request endpoint.
2. Each clinic gets a dedicated Twilio number → same Vapi org, different tenant config at runtime.
3. Test locally: `ngrok http 3001` + `vapi listen -forward-to localhost:3001/api/vapi/webhook`.

3.5 Phase E: Decommission custom phone orchestration (Day 5)

1. Remove ConversationRelay WebSocket scaffold from `voice.js` (keep Twilio signature middleware if needed).
2. Keep web widget (`js/`) for demo/dashboard embed; phone traffic goes through Vapi only.
3. Move webhook dispatch from browser IndexedDB to server-side queue for phone sessions.

4 Multi-Tenancy Pattern

One Vapi account. Tenant isolation in **your** database:

Tool calls include `client_id` as static parameter—your webhook never guesses which calendar to touch.

5 Cost Stack (per minute, 2026)

Layer	Rate	Notes
Vapi platform	$0.05/min	Fixed orchestration fee
Deepgram STT	∼$0.004–0.02/min	Nova-2 streaming
LLM (GPT-4o mini)	∼$0.003–0.01/min	Token-based
ElevenLabs TTS	∼$0.01–0.07/min	Voice tier dependent
Twilio telephony	∼$0.008–0.02/min	US inbound
All-in realistic	**$0.18–0.33/min**	Premium stack higher

At 1,000 min/month per clinic: raw cost $180–330. Price subscriptions above this with margin.

6 Migration Safety

Because integration = config + webhooks only, switching Vapi → LiveKit later rewrites the runtime layer only. Tool endpoints, dashboard, and tenant DB remain unchanged.

7 Acceptance Criteria

- Inbound call to clinic number → correct tenant greeting
- "Book Tuesday 10am" → real Google Calendar freebusy + book
- Emergency keyword → `/api/emergency` logged + transfer offered
- Call end → transcript in dashboard within 30s
- No browser tab required for phone call processing